Turnitin Report

IsharaCV-- Submission documents final

Slot 2

Document Details

Submission ID

trn:oid:::3618:136759213

Submission Date

Apr 27, 2026, 2:26 PM GMT+5

Download Date

Apr 27, 2026, 2:28 PM GMT+5

File Name

IsharaCV-- Submission documents final.docx

File Size

2.7 MB

11 Pages

757 Words

4,622 Characters

0% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

0 Not Cited or Quoted 0%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
0% Publications
0% Submitted works (Student Papers)

Match Groups

- **0** Not Cited or Quoted 0%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
0% Publications
0% Submitted works (Student Papers)

National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

IsharaCV

Semester Project

Object Oriented Programing

By

Bilal Ghazi

&

Hassaan Ali

Problem Statement:

In modern computing devices interaction with humans depends heavily upon input devices which require physical contact, while being effective this is a problem where physical touch is inconvenient and a limitation, or sometimes the users' hands are occupied with another task and multitasking is required

In addition, the use of a third party device feels less direct and lacks natural interaction, which feels less intuitive. And *there is a growing need for a seamless more natural way of communication*.

Solution Proposal:

To address this problem, we have come up with **IsharaCV** – A computer vision based Gesture to input translator which enables humans to communicate with their computer directly eliminating the need for external input devices

IsharaCV uses a real time feedback camera to recognize pre-mapped hand gestures to navigate the computer. It detects a gesture translates it into conventional computer based input and performs the required task

By eliminating the need for input devices **IsharaCV** provides:

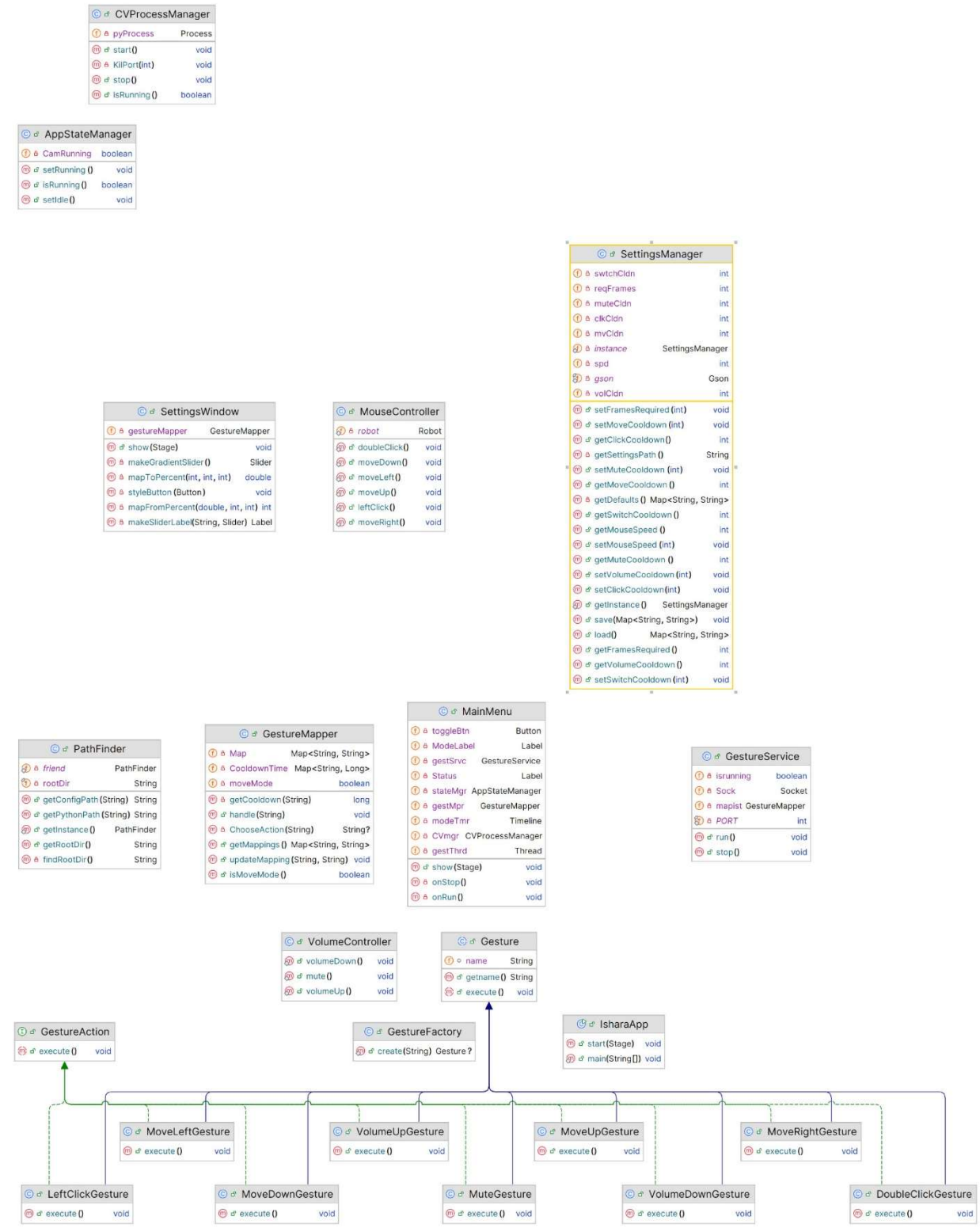
- A contactless UI
- Improved accessibility for users with physical limitations
- An improved and intuitive way to communicate with the computer
- Flexibility in environments where conventional input devices aren't practical

Object Oriented Programming usage:

Through this project we've demonstrated how Object Oriented programing can help us solve everyday challenges by finding and implementing new solutions

National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

UML



System Architecture:

The App follows a hybrid architecture utilizing both Python and Java to deliver hands-free desktop control.

- Python / mediapipe handles webcam gesture recognition.
- Java utilizes JNA and Robot to carry out desktop actions based on the received signal.
- The Python CV server communicates with the Java app using a TCP socket on the device.
- JavaFx utilized for GUI.

The directory itself is organized in a standardized pattern, first separating Java and Python files, then further dividing Java files into src, lib , and bin folders.

The src folder is further partitioned to separate and group together Java class files based on similarity in functions

Features:

Primary features include:

- Minimalistic and convenient gui.
- Live MainMenu status and mode indicators.
- Start/Stop toggle to run CV engine at will.
- 2 distinct modes, “Volume” and “Move”.
- Precise volume control; Up, Down, Mute.
- 4D mouse movement with the ability to Click and Double Click
- Seamless switch between modes with a gesture
- Intuitive settings window with a Tuning and Instructions Tab
- Detailed tuning options for both modes ie; Mouse speed, Detection sensitivity
- Instructions tab with gestures and their mapped functions for beginners.

Classes used:

Core system classes:

- CVProcessManager
  Responsible for starting and killing Python Engine.
- AppStateManager
  Tracks whether CV engine is running or Idle only.
- SettingsManager
  Manages setting related tasks
- IsharaApp
  The entry point to the app, creates core objects and forwards them to MainMenu.

Communication Layer:

- GestureService
  Receives names strings constantly from the server on a separate thread and forwards them to GestureMapper

Gesture Handling

- Gesture (base/abstract class)
  Abstract class, Parent class of all gestures
- GestureFactory
  Receives action string from Mapper and constructs and returns the corresponding Gesture subclass Object.
- GestureMapper
  Maps received strings from GestureService to appropriate actions while checking cooldowns.

Gestures

- MoveLeftGesture

- MoveRightGesture
- MoveUpGesture
- MoveDownGesture
- LeftClickGesture
- DoubleClickGesture
- VolumeUpGesture
- VolumeDownGesture
- MuteGesture

Action / Interface Layer

- GestureAction
  Interface, defines execute() method for all gestures

Controllers

- MouseController
  Uses Robot class to perform mouse functions.
- VolumeController
  Uses JNA to perform volume related functions.

UI

- MainMenuTest
  First Gui class, primary control centre.
- SettingsWindow
  Second Gui class, contains instructions and tuning controls.

Utility

- Pathfinder
  Finds the app's root directory (IsharaApp) at runtime.

(it is worth noting only classes native to this project are mentioned)

App Workflow:

User side:

User clicks IsharaApp.exe-> Mainmenu opens-> click start-> Python engine starts up

->make gestures to perform functions.

BackEnd:

Startup;

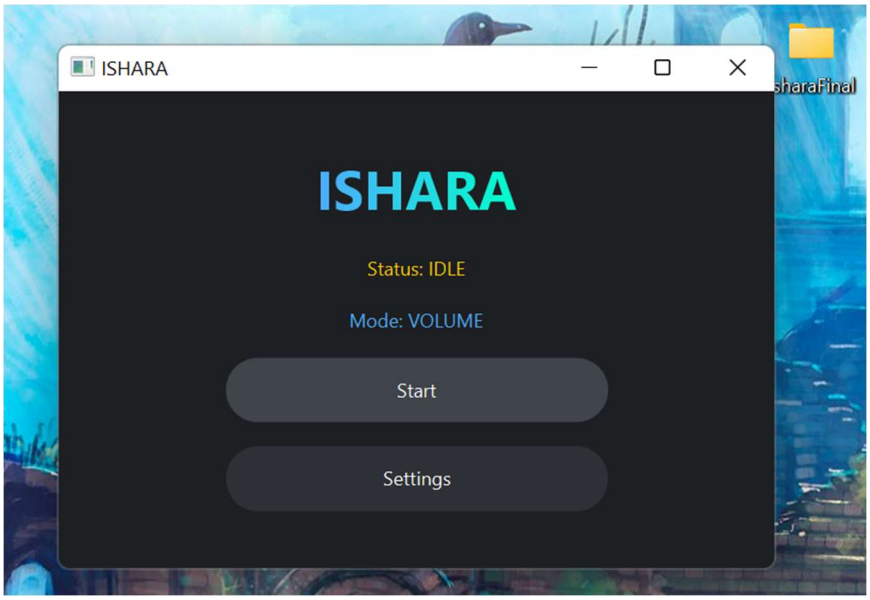
Start pressed ->MainMenu called-> Pathfinder called and finds root directory and returns full path to server-> GestureServer reads json for settings->mediapipe model loaded->Python clears port 6767 and connects-> GestureService is created in a separate thread-> Status changes to running.

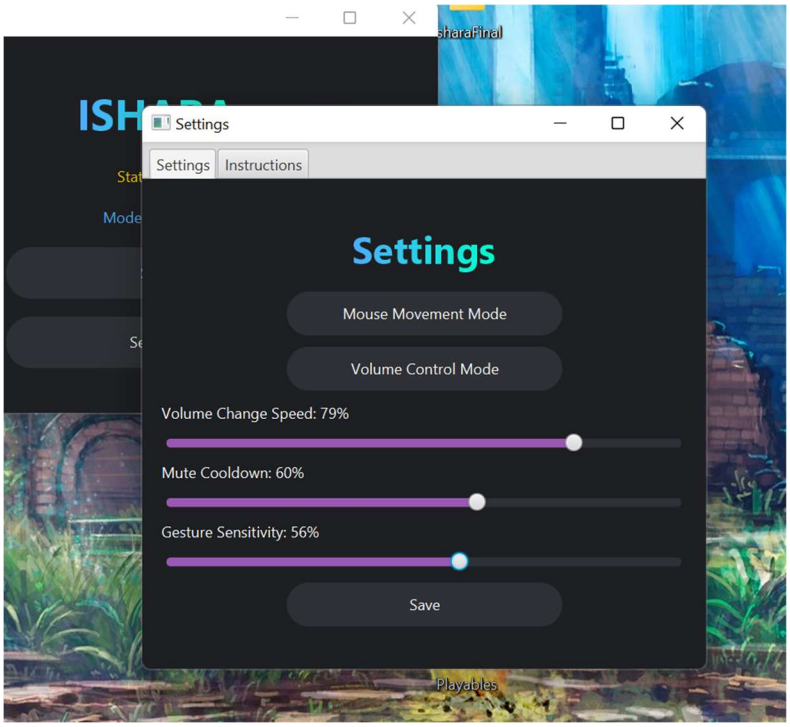
Gesture Processing:

Python reads webcam frames at 30fps-> GestureServer checks landmark positions-> If gesture persists for a number of frames its NAME is passed through socket-> GestureService receives NAME, processes it and forwards to mapper->Checks cooldown and maps NAME to corresponding ACTION forwarding to GestureFactory-> GestureFactory creates subclass object->Object calls execute() polymorphically -> corresponding controller method performs desktop action.

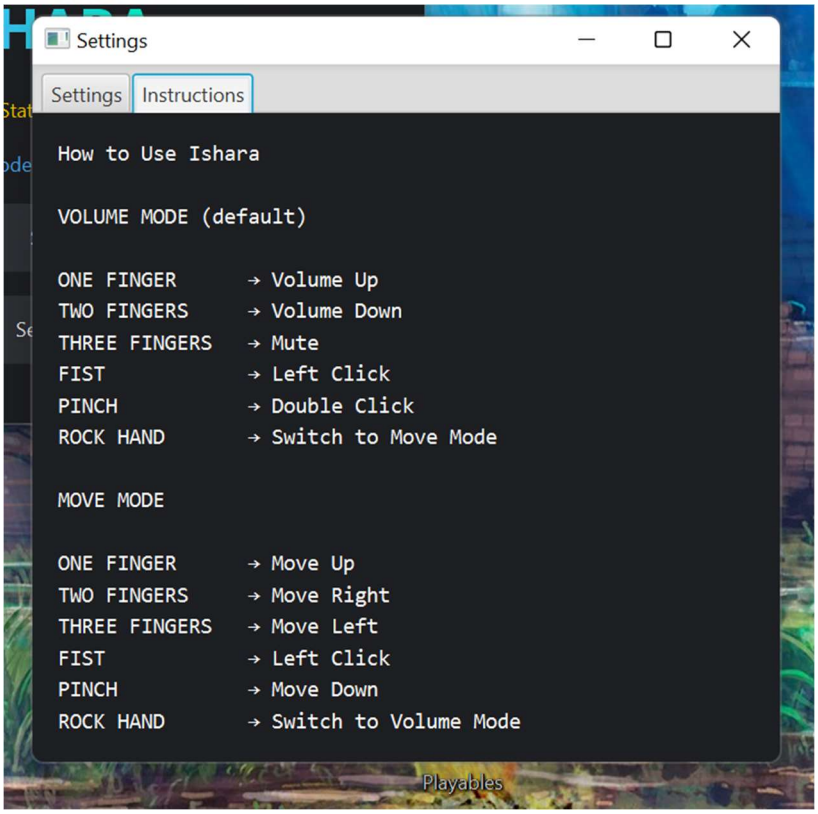
National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science

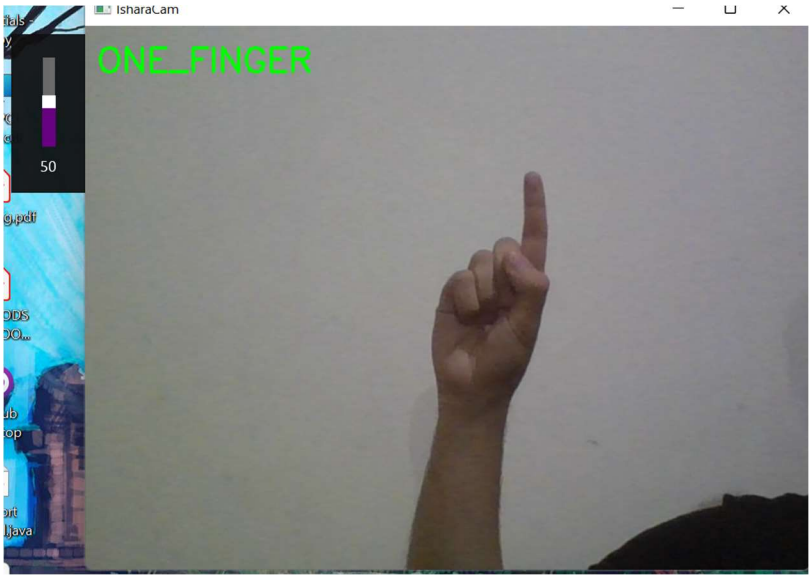
Screen Shots:



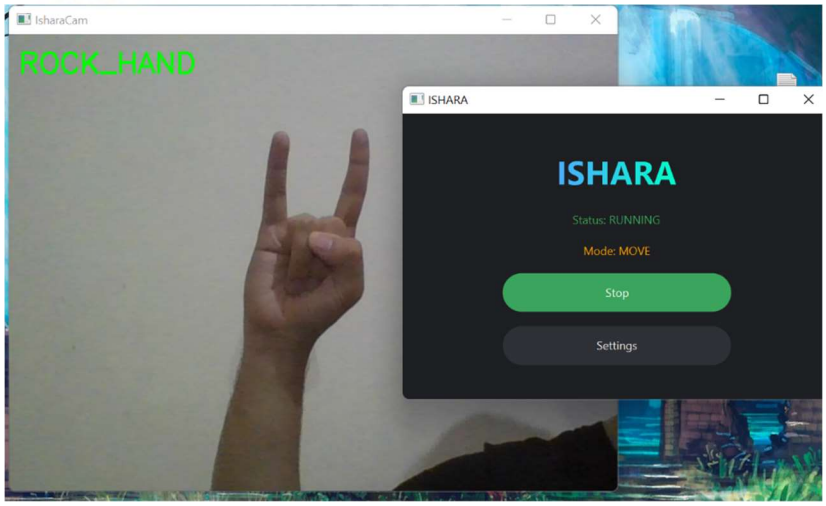


National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science





National University of Sciences and Technology (NUST)
School of Electrical Engineering and Computer Science



Github Repo Link:

https://github.com/BilalGh0zi/IsharaCV.git

Future Improvements:

- Linux support
- Two handed gestures
- Optional light frame on screen to counter low light scenarios
- More rigorous detection process